System Design, Volume 2

Consensus, storage internals, and the class drills as working code.

Volume 1 is the city map and the class blueprint. This volume opens the machines those maps assume: how a cluster picks one leader, how a disk remembers a write after a crash, and how the drills from volume 1 look as code. Every code file in the second half was run by `python test_volume2.py`. The run printed `all tests passed`.

The code lives next to this book, in `C:\Users\Admin\Documents\SystemDesignGuide\volume2\code`. You can read it here offline and run it there.

Contents

1. Where this fits
2. Clocks and causality
3. Leases, leaders, and fencing tokens
4. Raft
5. Paxos
6. Gossip and Merkle trees
7. The write-ahead log and pages
8. B-trees and LSM trees
9. MVCC, locks, and snapshots
10. What to say in a design
11. Working code

1. Where this fits

Volume 1 said “a follower becomes the leader” and “a row update is atomic.” Those sentences hide real machinery. You do not implement Raft inside a URL shortener. You do need to know what you are buying when you pick a database that claims a leader, a quorum, or snapshot reads. The same is true of the class drills: a checklist is finished when the code’s tests match the rules.

2. Clocks and causality

Machines do not share a perfect wall clock. A timestamp from server A can be earlier than a timestamp from server B even when B's event happened first. Designs that sort events by wall time alone will sometimes lie. Network time sync keeps clocks close. It does not make them a source of order.

Lamport clocks

Each process keeps an integer. On a local event it adds 1. When it sends a message it includes the integer. When it receives a message it sets its integer to one more than the maximum of its own value and the value in the message.

If event A happened before event B in the sense "A could have caused B" (same process in order, or a message from A's process reached B's process), then A's Lamport time is less than B's. The other direction is not guaranteed. A smaller number does not prove A caused B. Lamport clocks give you a total order you can sort, with process id as a tiebreak. They do not tell you when two events were independent.

Vector clocks

A vector clock is one integer per process. On a local event, a process adds 1 to its own slot. On a send, it includes the whole vector. On receive, it takes the max of each slot, then adds 1 to its own slot.

Compare two vectors. If every slot of A is less than or equal to B, and at least one slot is strictly less, A happened before B. If neither vector is less than or equal to the other, the events are **concurrent**. Concurrent means "neither could have known about the other," which is the moment a design needs a merge rule: last writer wins, keep both versions, or ask the user.

The code in `clocks.py` implements both clocks and the comparison. A shopping cart that accepts offline edits on two phones is the usual story. Each phone's vector shows the server the edits were concurrent, so it must merge the carts instead of letting one timestamp erase the other.

3. Leases, leaders, and fencing tokens

A **lease** is a lock with an expiry. The holder may act until the expiry. Everyone else waits. Leases are how a leader promises “I am the only writer until time T.” Followers refuse to elect a new leader until the lease has expired. That promise depends on a bound on clock skew: the holder must stop at expiry minus the maximum skew, so it does not write after others believe the lease is over.

A lease can still go wrong. The holder pauses (a long garbage-collection stop, a stalled disk) past the expiry. A new holder starts. The old holder wakes up and writes. Both believed they were the only writer.

A **fencing token** fixes the wake-up. Every successful acquire returns a number that only increases. The storage layer remembers the highest token it has accepted and rejects any write with a smaller token. The old holder’s late write carries the old number and is refused. The lock alone is not the safety. The token check on the write is the safety.

`LeaseLock` and `FencedStore` in `clocks.py` are this scene in a few lines. The test acquires a lease, waits it out, acquires a newer token, writes with the new token, and shows the old token cannot overwrite the value.

When a design says “we take a distributed lock in Redis,” the follow-up question is: what fencing token does the database check? A lock that can expire without a token is a performance hint, not a correctness tool.

4. Raft

Raft is a protocol that keeps a replicated log. Every server applies the same commands in the same order. That is the usual way a database gets one leader and identical replicas. The paper splits the problem into leader election, log replication, and safety.

Roles and terms

A server is a **follower**, a **candidate**, or a **leader**. Time is divided into **terms**, which are integers that only increase. Each term has at most one leader. A term can also end with no leader if an election splits the votes. The term number is on every message. A server that sees a higher term steps down and records it.

Election

Followers expect a heartbeat from the leader. The wait is an election timeout, randomized so two followers rarely time out together. If the heartbeat is missing, the follower becomes a candidate, adds 1 to its term, votes for itself, and asks the others for votes.

A follower grants a vote when the candidate's term is at least as new as its own, it has not voted for someone else in this term, and the candidate's log is at least as fresh as its own. Fresh means: the last entry's term is higher, or the term is equal and the log is at least as long. This rule is the one that stops an empty server from winning an election and erasing committed data.

A candidate that receives votes from a majority becomes leader and immediately sends heartbeats. A candidate that sees a higher term becomes a follower. A split vote means the timeouts fire again in a new term. Random timeouts make the retry likely to produce one winner.

Log replication

A client sends a command to the leader. The leader appends it to its log as an entry carrying the current term, then sends **AppendEntries** to the followers. The message includes the index and term of the entry just before the new ones. A follower rejects the message if that previous entry does not match, because the logs have diverged. The leader then retries from further back until the follower finds a matching prefix, deletes the conflicting tail, and appends the leader's tail.

An entry is **committed** when it is on a majority of servers and at least one entry from the leader's current term is committed with it. Raft commits an entry from an older term only as a consequence of committing a newer one. That detail prevents a later leader from erasing something another leader had already shown to a client. After commit, each server applies the entry to its state machine in log order. The client is told success only after the commit.

What a majority buys

Any two majorities overlap in at least one server. A new leader must have a log that a majority was willing to vote for, so its log contains every committed entry. That overlap is the whole safety argument in one sentence. It is also why a cluster of 3 tolerates 1 failure and a cluster of 5 tolerates 2. You count voting members. A server that is down does not vote. If a partition leaves a side without a majority, that side does not elect a leader and does not commit. The other side, if it has a majority, continues. Two leaders in one term would require two disjoint majorities, which cannot exist.

Membership changes

Adding a server is itself a log entry, not an out-of-band edit. Raft's joint-consensus approach commits a configuration that is the old set plus the new set. A decision in that phase needs a majority of the old set and a majority of the new set. A later entry switches to the new set only. Doing the change as one abrupt cut can, during the cut, let each side believe it has a majority.

The model in this book

`raft_model.py` runs these rules by method calls. `elect` asks for votes. Passing a short voter list is a partition: a candidate that can talk only to itself does not become leader. `propose` appends a command and replicates it. The test checks that all three nodes hold the command and that the commit index has moved. A third check shows a node whose log is newer refusing to vote for an empty candidate. There are no timers, no disk, and no retransmission loop. The rules you can see are the rules the tests lock in.

5. Paxos

Paxos solves the same problem: choose one value so a majority agrees, and every later reader sees that value. It is older than Raft. Raft's log is what you get when you run a chain of Paxos decisions with a stable leader. People explain Paxos as a single decision first.

Roles

Proposers suggest values. **Acceptors** vote. **Learners** hear the outcome. One machine can play all three roles. Acceptors are the quorum. You need a majority of acceptors.

Two phases

Proposal numbers are totally ordered. A proposer picks a number n higher than any it has used.

Prepare. The proposer sends n to the acceptors. An acceptor promises not to accept any proposal numbered below n. If it has already accepted something, it replies with the highest numbered value it accepted. If n is below a promise it already made, it rejects.

Accept. If a majority promised, the proposer sends an accept request. The value is not always the proposer's own idea. If any promise came back with an already accepted value, the proposer must use the one with the highest proposal number. That is how a value that was chosen stays chosen. Acceptors accept this proposal if they have not promised a higher number since.

The value is **chosen** when a majority of acceptors have accepted the same proposal. Learners can be told directly, or they can ask a majority and look for a value that appears in a majority of answers.

Why a later proposer cannot replace a chosen value

Suppose value V was accepted by a majority. Any later proposer must collect promises from a majority. Those two majorities share an acceptor. That acceptor reports V. The later proposer is required to propose V, not a new value. Higher numbers win the race to propose. They do not get to overwrite a choice.

Multi-Paxos and Raft

Choosing one value is one slot in a log. The next command is the next slot. If every slot runs both phases, you pay two round trips per command. **Multi-Paxos** elects a stable leader. The leader skips Prepare for later slots and only runs Accept, until it loses the leadership. Heartbeats, a log, and a commit index are the practical clothes on that idea. Raft writes those clothes down so two implementations do not guess different clothes. When a system says "Paxos" or "Raft," the property you are buying is the same: one ordered log, progress with a majority, and no committed entry thrown away.

6. Gossip and Merkle trees

Gossip spreads state the way a rumor spreads. Every few seconds a node picks a random peer and sends what it knows: membership, load, or the latest version number of a key. Each round roughly doubles the number of nodes that have heard. In a large cluster the news reaches everyone in a handful of rounds. Gossip tolerates lost messages because a later round repeats the news. It does not give you a committed log. Use it for membership lists, failure hints, and caches of metadata. Use Raft or Paxos when a write must be chosen once.

A **Merkle tree** is a tree of hashes. Each leaf is the hash of a key range. Each parent is the hash of its children. Two replicas compare roots. Equal roots mean equal data. Unequal roots mean they compare children and walk only the branches that differ. Anti-entropy repair then copies the missing keys. Cassandra-style stores and some backup systems use this so they do not ship the whole dataset to find a few drifted keys.

7. The write-ahead log and pages

A database keeps two pictures of your data. The **pages** are the tables as the next read wants them. The **write-ahead log** is an append-only record of what changed. The rule is the name: the log record is forced to stable storage before the changed page is. A crash can then lose a page and still rebuild it by replaying the log.

Recovery reads from the last **checkpoint**. A checkpoint is a note that says “pages up to this log position are on disk,” plus enough information to redo or undo in-flight transactions. Replay applies log records after that position. Records from transactions that committed are redone. Records from transactions that never committed are undone, if the system logged them before commit. After recovery the pages match the committed history.

`PageStore` is the crash window with the drama removed. `put` appends the log and then applies the page. `crash_after_log` appends the log and stops. The page still shows the old value. `recover` replays and the new value appears. Real engines add checksums, log segments, and group commit (several transactions forced in one disk write). The rule they are protecting is this one.

Group commit is why a durable transaction is slower than a memory write and much faster than one disk rotation per client. The leader waits until the log flush returns, then acknowledges the client. That wait is the latency you pay for “after a crash the row is still there.” Asynchronous replication, from volume 1, acknowledges before a follower has the log. The local WAL surviving is a different promise from “another building has the log.”

8. B-trees and LSM trees

Both structures store ordered keys. They spend their disk time differently.

B-trees

A B-tree is pages. Internal pages hold keys that route you left or right. Leaf pages hold the records, or pointers to them. A lookup is a short walk from the root, one disk read per level, and trees of millions of rows are only a few levels tall because each page holds hundreds of keys. Leaves are linked, so a range scan reads sequentially once it has found the start.

An insert that finds a full leaf **splits** the leaf into two and pushes a separator key into the parent. A parent that is full splits too, possibly up to the root. Updates to a B-tree are random I/O: the leaf for the next key may sit far from the leaf you just wrote. That random write is the cost. The benefit is a fast point read and a cheap range. The indexes in volume 1 are this structure unless the database tells you otherwise.

LSM trees

An LSM tree turns random writes into sequential ones. New writes enter a memory table, the **memtable**, often a balanced tree or a skip list. When the memtable is large enough it is frozen and written as a sorted file, an **SSTable**. The write to disk is sequential. Reads check the memtable first, then the SSTables from newest to oldest.

A key can appear in several files. The newest version wins. A delete is a **tombstone**, a marker that says "absent," so a later read does not return an older copy hiding in an older file. **Compaction** merges files, drops tombstones once they are no longer hiding anything, and throws away overwritten values. Compaction is the tax. You rewrite the same key several times over its life. That extra writing is **write amplification**. Checking several files on a read is **read amplification**.

A **Bloom filter** on each file answers "this key is definitely not in here" and lets the read skip the file. Volume 1 described the filter. Here is why the LSM needs it: without it, every read pays a disk check per file.

Two compaction styles show up in real systems. **Size-tiered** compaction merges files of similar size. Writes stay cheap and reads may touch more files. **Leveled** compaction keeps each level much larger than the one above and mostly one version of a key per level. Reads touch fewer files and compaction works harder. RocksDB, LevelDB, Cassandra, and many time-series stores are LSM engines. Traditional SQL engines are often B-trees. Newer SQL engines offer both.

`LsmTree` in `storage.py` is the shape: `put` and `delete` hit memory, `flush` freezes a run, `get` searches newest first, `compact` merges and drops tombstones. The test writes `old`, flushes, writes `new`, and reads `new`. Then it deletes, flushes, and compacts, and the key is gone with no tombstone left behind.

9. MVCC, locks, and snapshots

Volume 1 listed isolation levels. This is how a database actually provides them.

Two-phase locking

Under **two-phase locking** a transaction acquires locks in a growing phase and releases them in a shrinking phase. It does not acquire a new lock after it has released one. **Strict** two-phase locking holds write locks until commit or abort, so other transactions cannot read or overwrite uncommitted rows. Shared locks allow many readers. Exclusive locks allow one writer. Deadlocks are possible. The engine picks one transaction, aborts it, and the client retries. Locking makes writers wait on readers and readers wait on writers whenever they touch the same row.

MVCC

Multi-version concurrency control keeps old versions of a row. A writer does not overwrite the only copy. It creates a new version stamped with its transaction id and marks the previous version as ended by that same id. A reader carries a **snapshot**: the set of transactions that had committed when the reader began. A version is visible when its creator committed inside that snapshot (or the creator is the reader itself) and its deleter has not committed inside that snapshot. Readers do not take a write lock to see a stable picture. Writers do not wait for readers. The cost is storage of old versions and a **vacuum** job that deletes versions no snapshot can still see.

That snapshot is read committed or repeatable read, depending on when it is taken. A snapshot taken once at the start of the transaction is repeatable: later commits by others stay invisible. A snapshot refreshed on every statement is read committed. Serializable isolation needs an extra check, because two transactions can each read a snapshot and then write in a way that no serial order would allow. Engines detect that with predicate locks or by tracking the reads and aborting one writer. That abort is a normal outcome. The application retries the transaction.

`Mvcc` in `storage.py` is a snapshot reader. The test begins T1, writes `a=one`, begins an early reader before T1 commits, commits T1, then begins a later reader. The early snapshot does not see the row. The later snapshot does. T1 can read its own write. Nobody took a lock.

What you choose

Need	Mechanism that serves it
A read of many rows that match each other	One snapshot for the whole transaction
A seat or a balance that two writers must not both take	A conditional update or an exclusive lock, plus a unique key
Readers that stay fast while writers work	MVCC
A report that can see last Tuesday	Versions kept long enough, or a replica dedicated to reads

10. What to say in a design

You now have a precise sentence for each promise volume 1 left informal.

- “One leader” means a majority protocol, Raft or Multi-Paxos, with a term and a log. A side that lacks a majority does not take writes.
- “Failover” means a new election. Committed entries are on the new leader because of the vote’s log check. Uncommitted entries on the old leader can disappear.
- “The write is durable” means the local write-ahead log was forced. “The write survives a building failure” means a follower acknowledged it. Those are different waits.
- “The lock is safe across expiry” means the write carries a fencing token and storage rejects smaller tokens.
- “Readers do not block writers” means MVCC snapshots, and a serializable conflict ends in a retry.
- “This store is write-heavy” is a reason to expect an LSM engine, compaction, and Bloom filters. “This store is range-heavy” is a reason to expect a B-tree.
- Two offline editors need a vector clock or a log of operations. A pair of wall-clock timestamps will silently drop one of the edits.

11. Working code

The rest of this book is the source that the tests execute. Read each file against the rule named above it. Run them from the code folder:

```
cd C:\Users\Admin\Documents\SystemDesignGuide\volume2\code
python test_volume2.py
```

The files are the drills from volume 1, plus the models from this volume: LRU cache, token bucket, base62, TinyLink, parking lot, seat holds, elevator, logger, ledger, Raft model, write-ahead log, LSM, MVCC, clocks, and the fencing token.

clocks.py

Clocks and the fencing token.

```
"""Lamport clocks, vector clocks, and a lease lock with a fencing token."""

from __future__ import annotations


class LamportClock:
    def __init__(self) -> None:
        self.time = 0

    def tick(self) -> int:
        self.time += 1
        return self.time

    def receive(self, incoming: int) -> int:
        self.time = max(self.time, incoming) + 1
        return self.time


class VectorClock:
    def __init__(self, node_id: str, nodes: list[str]) -> None:
        self.node_id = node_id
        self.time = {node: 0 for node in nodes}

    def tick(self) -> dict[str, int]:
        self.time[self.node_id] += 1
        return dict(self.time)

    def receive(self, incoming: dict[str, int]) -> dict[str, int]:
        for node, value in incoming.items():
            self.time[node] = max(self.time[node], value)
        self.time[self.node_id] += 1
        return dict(self.time)


def happened_before(left: dict[str, int], right: dict[str, int]) -> bool:
    """True when left is a causal ancestor of right."""
    less_or_equal = all(left[key] <= right[key] for key in left)
    strictly_less = any(left[key] < right[key] for key in left)
    return less_or_equal and strictly_less


def concurrent(left: dict[str, int], right: dict[str, int]) -> bool:
    return not happened_before(left, right) and not happened_before(right, left)


class StaleFencingToken(Exception):
    pass


class LeaseLock:
    """The lock can expire. The fencing token is what makes a late writer harmless."""

    def __init__(self) -> None:
        self.holder: str | None = None
        self.expires_at = 0.0
        self.token = 0

    def acquire(self, client: str, now: float, ttl: float) -> int | None:
        if self.holder not in (None, client) and now < self.expires_at:
            return None
        self.token += 1
        self.holder = client
        self.expires_at = now + ttl
        return self.token
```

```
class FencedStore:
    def __init__(self) -> None:
        self.seen = 0
        self.value: str | None = None

    def write(self, token: int, value: str) -> None:
        if token < self.seen:
            raise StaleFencingToken(token)
        self.seen = token
        self.value = value
```

raft_model.py

Raft election, append, and commit, by direct calls.

```
"""In-process model of Raft's rules.

This is the election, vote, append, and commit logic from the Raft paper,
run by direct calls instead of timers and a network. It exists so you can
see a majority, a rejected stale vote, and a committed log entry.
It is not a production Raft library.
"""

from __future__ import annotations

from dataclasses import dataclass


@dataclass
class Entry:
    term: int
    command: str


class RaftNode:
    def __init__(self, node_id: str) -> None:
        self.node_id = node_id
        self.current_term = 0
        self.voted_for: str | None = None
        self.log: list[Entry] = []
        self.commit_index = -1
        self.role = "follower"
        self.leader_id: str | None = None

    def last_index(self) -> int:
        return len(self.log) - 1

    def last_term(self) -> int:
        if not self.log:
            return 0
        return self.log[-1].term

    def request_vote(self, term: int, candidate_id: str, last_log_index: int, last_log_term: int) -> tuple[int,
bool]:
        if term < self.current_term:
            return self.current_term, False
        if term > self.current_term:
            self._become_follower(term)
        log_ok = self._at_least_as_fresh(last_log_index, last_log_term)
        if log_ok and self.voted_for in (None, candidate_id):
            self.voted_for = candidate_id
            return self.current_term, True
        return self.current_term, False

    def append_entries(
        self,
        term: int,
        leader_id: str,
        prev_index: int,
        prev_term: int,
        entries: list[Entry],
        leader_commit: int,
    ) -> tuple[int, bool]:
        if term < self.current_term:
            return self.current_term, False
        if term > self.current_term:
            self._become_follower(term)
        self.role = "follower"
        self.leader_id = leader_id
```

```
        if prev_index >= 0:
            if prev_index >= len(self.log) or self.log[prev_index].term != prev_term:
                return self.current_term, False
        for offset, entry in enumerate(entries):
            index = prev_index + 1 + offset
            if index < len(self.log):
                if self.log[index].term != entry.term:
                    del self.log[index:]
                    self.log.append(entry)
            else:
                self.log.append(entry)
        if leader_commit > self.commit_index:
            self.commit_index = min(leader_commit, len(self.log) - 1)
        return self.current_term, True

    def _at_least_as_fresh(self, last_log_index: int, last_log_term: int) -> bool:
        mine = self.last_term()
        if last_log_term != mine:
            return last_log_term > mine
        return last_log_index >= self.last_index()

    def _become_follower(self, term: int) -> None:
        self.current_term = term
        self.role = "follower"
        self.voted_for = None
        self.leader_id = None


class Cluster:
    def __init__(self, node_ids: list[str]) -> None:
        self.nodes = {node_id: RaftNode(node_id) for node_id in node_ids}

    def elect(self, candidate_id: str, voters: list[str] | None = None) -> bool:
        """voters limits who answers. A partition is a short voter list."""
        candidate = self.nodes[candidate_id]
        candidate.current_term += 1
        candidate.role = "candidate"
        candidate.voted_for = candidate_id
        candidate.leader_id = None
        votes = 1
        group = self.nodes if voters is None else {node_id: self.nodes[node_id] for node_id in voters}
        for other in group.values():
            if other is candidate:
                continue
            term, granted = other.request_vote(
                candidate.current_term,
                candidate_id,
                candidate.last_index(),
                candidate.last_term(),
            )
            if term > candidate.current_term:
                candidate._become_follower(term)
                return False
            if granted:
                votes += 1
        majority = len(self.nodes) // 2 + 1
        if votes >= majority:
            candidate.role = "leader"
            candidate.leader_id = candidate_id
            return True
        candidate.role = "follower"
        return False

    def propose(self, leader_id: str, command: str) -> int:
        leader = self.nodes[leader_id]
        if leader.role != "leader":
            raise RuntimeError("not the leader")
        leader.log.append(Entry(leader.current_term, command))
        self.replicate(leader_id)
        return leader.commit_index
```

```
    def replicate(self, leader_id: str) -> None:
        leader = self.nodes[leader_id]
        if leader.role != "leader":
            return
        matched: dict[str, int] = {leader.node_id: leader.last_index()}
        for other in self.nodes.values():
            if other is leader:
                continue
            prev = common_prefix(leader, other)
            entries = leader.log[prev + 1 :]
            prev_term = 0 if prev < 0 else leader.log[prev].term
            term, ok = other.append_entries(
                leader.current_term,
                leader.node_id,
                prev,
                prev_term,
                entries,
                leader.commit_index,
            )
            if term > leader.current_term:
                leader._become_follower(term)
                return
            if ok:
                matched[other.node_id] = other.last_index()
        self._advance_commit(leader, matched)
        for other in self.nodes.values():
            if other is leader or leader.role != "leader":
                continue
            prev = other.last_index()
            prev_term = 0 if prev < 0 else other.log[prev].term
            other.append_entries(leader.current_term, leader.node_id, prev, prev_term, [], leader.commit_index)

    def _advance_commit(self, leader: RaftNode, matched: dict[str, int]) -> None:
        majority = len(self.nodes) // 2 + 1
        for index in range(leader.last_index(), leader.commit_index, -1):
            if leader.log[index].term != leader.current_term:
                continue
            copies = sum(1 for reached in matched.values() if reached >= index)
            if copies >= majority:
                leader.commit_index = index
                return


def common_prefix(leader: RaftNode, follower: RaftNode) -> int:
    index = min(len(leader.log), len(follower.log)) - 1
    while index >= 0:
        if leader.log[index].term == follower.log[index].term:
            return index
        index -= 1
    return -1
```

storage.py

Write-ahead log, LSM runs, and snapshot reads.

```
"""Small working models of a write-ahead log, an LSM tree, and MVCC."""

from __future__ import annotations


class PageStore:
    """Log the change, then touch the page. Recovery replays anything not applied."""

    def __init__(self) -> None:
        self.pages: dict[str, str] = {}
        self.wal: list[tuple[str, str]] = []
        self.applied = 0

    def put(self, key: str, value: str) -> None:
        self.wal.append((key, value))
        self._apply_next()

    def crash_after_log(self, key: str, value: str) -> None:
        """The log has the write. The page does not. This is a crash window."""
        self.wal.append((key, value))

    def recover(self) -> None:
        while self.applied < len(self.wal):
            self._apply_next()

    def _apply_next(self) -> None:
        key, value = self.wal[self.applied]
        self.pages[key] = value
        self.applied += 1


class LsmTree:
    """Writes land in memory. Flushes become sorted runs. Reads start at the newest run."""

    def __init__(self) -> None:
        self.mem: dict[str, str | None] = {}
        self.runs: list[dict[str, str | None]] = []

    def put(self, key: str, value: str) -> None:
        self.mem[key] = value

    def delete(self, key: str) -> None:
        self.mem[key] = None

    def flush(self) -> None:
        if not self.mem:
            return
        self.runs.append(self.mem)
        self.mem = {}

    def get(self, key: str) -> str | None:
        if key in self.mem:
            return self.mem[key]
        for run in reversed(self.runs):
            if key in run:
                return run[key]
        return None

    def compact(self) -> None:
        merged: dict[str, str | None] = {}
        for run in self.runs:
            merged.update(run)
        merged.update(self.mem)
        self.runs = [{key: value for key, value in merged.items() if value is not None}]
```

```
        self.mem = {}


class Mvcc:
    """Readers see a snapshot. Writers add a new version. They do not block each other."""

    def __init__(self) -> None:
        self._next = 1
        self.committed: set[int] = set()
        self.versions: dict[str, list[tuple[int, int | None, str | None]]] = {}

    def begin(self) -> tuple[int, frozenset[int]]:
        txn = self._next
        self._next += 1
        return txn, frozenset(self.committed)

    def commit(self, txn: int) -> None:
        self.committed.add(txn)

    def write(self, txn: int, key: str, value: str | None) -> None:
        chain = self.versions.setdefault(key, [])
        for index, (xmin, xmax, _) in enumerate(chain):
            if xmax is None and self._visible_creator(xmin, txn, frozenset(self.committed)):
                chain[index] = (xmin, txn, chain[index][2])
        chain.append((txn, None, value))

    def read(self, txn: int, snapshot: frozenset[int], key: str) -> str | None:
        for xmin, xmax, value in reversed(self.versions.get(key, [])):
            if self._visible(xmin, xmax, txn, snapshot):
                return value
        return None

    def _visible(self, xmin: int, xmax: int | None, txn: int, snapshot: frozenset[int]) -> bool:
        created = self._visible_creator(xmin, txn, snapshot)
        deleted = xmax is not None and (xmax == txn or xmax in snapshot)
        return created and not deleted

    def _visible_creator(self, xmin: int, txn: int, snapshot: frozenset[int]) -> bool:
        return xmin == txn or xmin in snapshot
```

lru.py

Hash map plus a doubly linked list.

```
"""Least-recently-used cache. get and put are O(1)."""

from __future__ import annotations


class _Node:
    def __init__(self, key: object, value: object) -> None:
        self.key = key
        self.value = value
        self.prev: _Node | None = None
        self.next: _Node | None = None


class LruCache:
    def __init__(self, capacity: int) -> None:
        if capacity < 1:
            raise ValueError("capacity must be at least 1")
        self.capacity = capacity
        self._map: dict[object, _Node] = {}
        self._head = _Node(None, None)
        self._tail = _Node(None, None)
        self._head.next = self._tail
        self._tail.prev = self._head

    def __len__(self) -> int:
        return len(self._map)

    def get(self, key: object) -> object:
        node = self._map.get(key)
        if node is None:
            return None
        self._move_to_front(node)
        return node.value

    def put(self, key: object, value: object) -> None:
        node = self._map.get(key)
        if node is not None:
            node.value = value
            self._move_to_front(node)
            return
        node = _Node(key, value)
        self._map[key] = node
        self._insert_front(node)
        if len(self._map) > self.capacity:
            evicted = self._tail.prev
            assert evicted is not None and evicted is not self._head
            self._unlink(evicted)
            del self._map[evicted.key]

    def _insert_front(self, node: _Node) -> None:
        nxt = self._head.next
        assert nxt is not None
        node.prev = self._head
        node.next = nxt
        self._head.next = node
        nxt.prev = node

    def _unlink(self, node: _Node) -> None:
        assert node.prev is not None and node.next is not None
        node.prev.next = node.next
        node.next.prev = node.prev
        node.prev = None
        node.next = None
```

```
    def _move_to_front(self, node: _Node) -> None:
        self._unlink(node)
        self._insert_front(node)
```

rate_limiter.py

Token bucket with an injected clock.

```
"""Token bucket. Time is injected so tests do not sleep."""

from __future__ import annotations


class TokenBucket:
    def __init__(self, capacity: float, refill_per_second: float, now: float) -> None:
        if capacity <= 0 or refill_per_second < 0:
            raise ValueError("capacity must be positive and refill must be >= 0")
        self.capacity = float(capacity)
        self.refill_per_second = float(refill_per_second)
        self.tokens = float(capacity)
        self.last = float(now)

    def _refill(self, now: float) -> None:
        elapsed = now - self.last
        if elapsed < 0:
            elapsed = 0
        self.tokens = min(self.capacity, self.tokens + elapsed * self.refill_per_second)
        self.last = now

    def allow(self, now: float, cost: float = 1.0) -> bool:
        self._refill(now)
        if self.tokens < cost:
            return False
        self.tokens -= cost
        return True


class RateLimiter:
    """One bucket per caller key. A lock per key keeps two threads honest."""

    def __init__(self, capacity: float, refill_per_second: float) -> None:
        self.capacity = capacity
        self.refill_per_second = refill_per_second
        self._buckets: dict[str, TokenBucket] = {}

    def allow(self, key: str, now: float) -> bool:
        bucket = self._buckets.get(key)
        if bucket is None:
            bucket = TokenBucket(self.capacity, self.refill_per_second, now)
            self._buckets[key] = bucket
        return bucket.allow(now)
```

codec.py

Base62 encode and decode.

```
"""Base62. Same number always becomes the same short code."""

ALPHABET = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"


def encode(number: int) -> str:
    if number < 0:
        raise ValueError("number must be >= 0")
    if number == 0:
        return ALPHABET[0]
    chars: list[str] = []
    while number:
        number, remainder = divmod(number, 62)
        chars.append(ALPHABET[remainder])
    return "".join(reversed(chars))


def decode(code: str) -> int:
    if not code:
        raise ValueError("empty code")
    number = 0
    for char in code:
        index = ALPHABET.find(char)
        if index < 0:
            raise ValueError(f"bad character: {char}")
        number = number * 62 + index
    return number
```

tinylink.py

Create, open, expiry, unique codes, and a retry token.

```
"""TinyLink rules, with an in-memory notebook that enforces unique keys."""

from __future__ import annotations

from dataclasses import dataclass

from codec import decode, encode


class Conflict(Exception):
    pass


class NotFound(Exception):
    pass


class Expired(Exception):
    pass


@dataclass
class Link:
    id: int
    code: str
    long_url: str
    created_at: float
    expires_at: float | None
    click_count: int
    request_token: str | None


@dataclass
class CreateCommand:
    long_url: str
    request_token: str
    custom_code: str | None = None
    expires_at: float | None = None


class MemoryRepo:
    def __init__(self) -> None:
        self.by_code: dict[str, Link] = {}
        self.by_token: dict[str, Link] = {}

    def find_by_code(self, code: str) -> Link | None:
        return self.by_code.get(code)

    def find_by_token(self, token: str) -> Link | None:
        return self.by_token.get(token)

    def insert(self, link: Link) -> None:
        if link.code in self.by_code:
            raise Conflict(link.code)
        if link.request_token and link.request_token in self.by_token:
            raise Conflict(link.request_token)
        self.by_code[link.code] = link
        if link.request_token:
            self.by_token[link.request_token] = link


class MemoryCache:
    def __init__(self) -> None:
        self.store: dict[str, Link] = {}
```

```
        self.fail = False

    def get(self, code: str) -> Link | None:
        if self.fail:
            raise RuntimeError("cache down")
        return self.store.get(code)

    def put(self, code: str, link: Link) -> None:
        if self.fail:
            return
        self.store[code] = link


class IdSource:
    def __init__(self, start: int = 1) -> None:
        self._next = start

    def next_id(self) -> int:
        value = self._next
        self._next += 1
        return value


class ClickQueue:
    def __init__(self) -> None:
        self.events: list[str] = []

    def publish(self, code: str) -> None:
        self.events.append(code)


def valid_url(url: str) -> bool:
    if len(url) > 2048:
        return False
    return url.startswith("http://") or url.startswith("https://")


class LinkService:
    def __init__(self, repo: MemoryRepo, cache: MemoryCache, ids: IdSource, clicks: ClickQueue) -> None:
        self.repo = repo
        self.cache = cache
        self.ids = ids
        self.clicks = clicks

    def create(self, command: CreateCommand, now: float) -> Link:
        if not valid_url(command.long_url):
            raise ValueError("invalid url")
        existing = self.repo.find_by_token(command.request_token)
        if existing is not None:
            return existing
        if command.custom_code:
            code = command.custom_code
            link_id = self.ids.next_id()
        else:
            link_id = self.ids.next_id()
            code = encode(link_id)
        link = Link(
            id=link_id,
            code=code,
            long_url=command.long_url,
            created_at=now,
            expires_at=command.expires_at,
            click_count=0,
            request_token=command.request_token,
        )
        try:
            self.repo.insert(link)
        except Conflict:
            if command.custom_code:
                raise
            raise
```

```
        self.cache.put(code, link)
        return link

    def open(self, code: str, now: float) -> str:
        link = None
        try:
            link = self.cache.get(code)
        except RuntimeError:
            link = None
        if link is None:
            link = self.repo.find_by_code(code)
            if link is not None:
                try:
                    self.cache.put(code, link)
                except RuntimeError:
                    pass
        if link is None:
            raise NotFound(code)
        if link.expires_at is not None and link.expires_at <= now:
            raise Expired(code)
        self.clicks.publish(code)
        return link.long_url


def apply_click(repo: MemoryRepo, code: str) -> None:
    """Worker. Adding 1 on the stored row is safe to repeat only if the event id is new.

    This helper is the approximate-counter version: each delivery adds one.
    Callers that need exactly-once must dedupe event ids before calling it.
    """
    link = repo.find_by_code(code)
    if link is None:
        return
    link.click_count += 1


def shard_for(code: str, shard_count: int) -> int:
    """Range-style home: decode the code back to the id, then mod."""
    return decode(code) % shard_count
```

parking.py

Spot allocation, including a five-spot bus, and idempotent leave.

```
"""Parking lot. A bus takes five consecutive car spots. Leave is idempotent."""

from __future__ import annotations

from dataclasses import dataclass, field


class Full(Exception):
    pass


class UnknownTicket(Exception):
    pass


@dataclass
class Spot:
    spot_id: str
    size: str  # "M" motorcycle, "C" car
    free: bool = True
    ticket_id: str | None = None


@dataclass
class Vehicle:
    plate: str
    kind: str  # motorcycle, car, bus

    def spots_needed(self) -> int:
        return 5 if self.kind == "bus" else 1


@dataclass
class Ticket:
    ticket_id: str
    plate: str
    kind: str
    spot_ids: list[str]
    entry: float
    exit: float | None = None
    fee: int | None = None


@dataclass
class Row:
    spots: list[Spot]

    def allocate(self, vehicle: Vehicle, ticket_id: str) -> list[Spot] | None:
        if vehicle.kind == "bus":
            run: list[Spot] = []
            for spot in self.spots:
                if spot.free and spot.size == "C":
                    run.append(spot)
                    if len(run) == 5:
                        break
                else:
                    run = []
            if len(run) < 5:
                return None
            chosen = run
        else:
            preferred = ["M", "C"] if vehicle.kind == "motorcycle" else ["C"]
            chosen_one = None
            for size in preferred:
```

```
                for spot in self.spots:
                    if spot.free and spot.size == size:
                        chosen_one = spot
                        break
                if chosen_one is not None:
                    break
            if chosen_one is None:
                return None
            chosen = [chosen_one]
        for spot in chosen:
            spot.free = False
            spot.ticket_id = ticket_id
        return chosen


class HourlyFee:
    def __init__(self, rate: int) -> None:
        self.rate = rate

    def fee(self, entry: float, exit_time: float) -> int:
        hours = max(1, int((exit_time - entry + 3599) // 3600))
        return hours * self.rate


@dataclass
class ParkingLot:
    rows: list[Row]
    fees: HourlyFee
    tickets: dict[str, Ticket] = field(default_factory=dict)
    _seq: int = 0

    def park(self, vehicle: Vehicle, now: float) -> Ticket:
        self._seq += 1
        ticket_id = f"T{self._seq}"
        taken: list[Spot] = []
        for row in self.rows:
            got = row.allocate(vehicle, ticket_id)
            if got is not None:
                taken = got
                break
        if not taken:
            raise Full()
        ticket = Ticket(ticket_id, vehicle.plate, vehicle.kind, [s.spot_id for s in taken], now)
        self.tickets[ticket_id] = ticket
        return ticket

    def leave(self, ticket_id: str, now: float) -> int:
        ticket = self.tickets.get(ticket_id)
        if ticket is None:
            raise UnknownTicket(ticket_id)
        if ticket.exit is not None and ticket.fee is not None:
            return ticket.fee
        for row in self.rows:
            for spot in row.spots:
                if spot.spot_id in ticket.spot_ids:
                    spot.free = True
                    spot.ticket_id = None
        ticket.exit = now
        ticket.fee = self.fees.fee(ticket.entry, now)
        return ticket.fee
```

tickets.py

Conditional seat hold, confirm, expire, and two threads on one seat.

```
"""Seat holds. The conditional update is what makes two buyers safe."""

from __future__ import annotations

import threading
from dataclasses import dataclass, field


class Unavailable(Exception):
    pass


class BadState(Exception):
    pass


@dataclass
class Seat:
    seat_id: str
    state: str = "free"  # free, held, sold
    booking_id: str | None = None


@dataclass
class Booking:
    booking_id: str
    seat_ids: list[str]
    user_id: str
    state: str  # HELD, CONFIRMED, EXPIRED
    hold_expires_at: float


class Show:
    def __init__(self, seat_ids: list[str]) -> None:
        self.seats = {seat_id: Seat(seat_id) for seat_id in seat_ids}
        self.bookings: dict[str, Booking] = {}
        self._seq = 0
        self._lock = threading.Lock()

    def hold(self, seat_ids: list[str], user_id: str, now: float, ttl: float = 300.0) -> Booking:
        """Same shape as:

        UPDATE seats SET booking_id = ? WHERE seat_id IN (...) AND state = 'free'
        Commit only when every requested row changed.
        """
        with self._lock:
            for seat_id in seat_ids:
                seat = self.seats[seat_id]
                if seat.state != "free":
                    raise Unavailable(seat_id)
            self._seq += 1
            booking_id = f"B{self._seq}"
            for seat_id in seat_ids:
                seat = self.seats[seat_id]
                seat.state = "held"
                seat.booking_id = booking_id
            booking = Booking(booking_id, list(seat_ids), user_id, "HELD", now + ttl)
            self.bookings[booking_id] = booking
            return booking

    def confirm(self, booking_id: str, now: float) -> Booking:
        with self._lock:
            booking = self.bookings[booking_id]
            if booking.state == "CONFIRMED":
```

```
                return booking
            if booking.state != "HELD" or booking.hold_expires_at <= now:
                raise BadState(booking.state)
            for seat_id in booking.seat_ids:
                seat = self.seats[seat_id]
                if seat.booking_id != booking_id or seat.state != "held":
                    raise BadState(seat_id)
            for seat_id in booking.seat_ids:
                self.seats[seat_id].state = "sold"
            booking.state = "CONFIRMED"
            return booking

    def expire(self, now: float) -> int:
        with self._lock:
            expired = 0
            for booking in self.bookings.values():
                if booking.state == "HELD" and booking.hold_expires_at <= now:
                    for seat_id in booking.seat_ids:
                        seat = self.seats[seat_id]
                        if seat.booking_id == booking.booking_id and seat.state == "held":
                            seat.state = "free"
                            seat.booking_id = None
                    booking.state = "EXPIRED"
                    expired += 1
            return expired
```

elevator.py

LOOK: finish the current direction, then reverse.

```
"""One elevator. LOOK: finish this direction, then reverse."""

from __future__ import annotations


class Elevator:
    def __init__(self, floor: int = 0) -> None:
        self.floor = floor
        self.direction = "IDLE"
        self.up: set[int] = set()
        self.down: set[int] = set()
        self.doors_open = False

    def call(self, floor: int, direction: str) -> None:
        if direction == "UP":
            self.up.add(floor)
        elif direction == "DOWN":
            self.down.add(floor)
        else:
            raise ValueError(direction)
        self._wake()

    def choose(self, floor: int) -> None:
        if floor > self.floor:
            self.up.add(floor)
        elif floor < self.floor:
            self.down.add(floor)
        else:
            self.doors_open = True
        self._wake()

    def step(self) -> None:
        self.doors_open = False
        if self.direction == "IDLE":
            self._wake()
            if self.direction == "IDLE":
                return
        if self.direction == "UP":
            if self.floor in self.up:
                self.up.discard(self.floor)
                self.doors_open = True
                if not self.up:
                    self.direction = "DOWN" if self.down else "IDLE"
                return
            if self.up and self.floor < max(self.up):
                self.floor += 1
                return
            self.direction = "DOWN" if self.down else "IDLE"
            return
        if self.floor in self.down:
            self.down.discard(self.floor)
            self.doors_open = True
            if not self.down:
                self.direction = "UP" if self.up else "IDLE"
            return
        if self.down and self.floor > min(self.down):
            self.floor -= 1
            return
        self.direction = "UP" if self.up else "IDLE"

    def _wake(self) -> None:
        if self.direction != "IDLE":
            return
        if any(floor > self.floor for floor in self.up | self.down):
```

```
            self.direction = "UP"
        elif any(floor < self.floor for floor in self.up | self.down):
            self.direction = "DOWN"
        elif self.floor in self.up or self.floor in self.down:
            self.direction = "UP" if self.floor in self.up else "DOWN"
```

logger.py

Level filter. One failing appender does not stop the others.

```
"""Logger. Levels filter. A failing appender does not sink the others."""

from __future__ import annotations

from dataclasses import dataclass, field


LEVELS = {"DEBUG": 10, "INFO": 20, "WARN": 30, "ERROR": 40}


@dataclass
class LogEvent:
    level: str
    logger_name: str
    message: str
    context: dict[str, str] = field(default_factory=dict)


class ListAppender:
    def __init__(self) -> None:
        self.events: list[LogEvent] = []
        self.fail = False

    def append(self, event: LogEvent) -> None:
        if self.fail:
            raise RuntimeError("disk full")
        self.events.append(event)


class Logger:
    def __init__(self, name: str, min_level: str, appenders: list[ListAppender]) -> None:
        self.name = name
        self.min_level = min_level
        self.appenders = appenders
        self.dropped_errors = 0

    def log(self, level: str, message: str, **context: str) -> None:
        if LEVELS[level] < LEVELS[self.min_level]:
            return
        event = LogEvent(level, self.name, message, dict(context))
        for appender in self.appenders:
            try:
                appender.append(event)
            except Exception:
                self.dropped_errors += 1
```

ledger.py

Append-only expenses. Nets are a sum. Simplified debts are a display.

```
"""Splitwise-style ledger. History is append-only. Balances are a sum."""

from __future__ import annotations

from dataclasses import dataclass, field


class Unbalanced(Exception):
    pass


@dataclass
class Expense:
    expense_id: str
    payer_id: str
    shares: dict[str, int]


@dataclass
class Settlement:
    from_user: str
    to_user: str
    amount: int


@dataclass
class GroupLedger:
    expenses: list[Expense] = field(default_factory=list)
    settlements: list[Settlement] = field(default_factory=list)
    _seq: int = 0

    def add_expense(self, payer_id: str, shares: dict[str, int]) -> Expense:
        if sum(shares.values()) <= 0:
            raise Unbalanced("empty")
        if any(amount < 0 for amount in shares.values()):
            raise Unbalanced("negative share")
        self._seq += 1
        expense = Expense(f"E{self._seq}", payer_id, dict(shares))
        self.expenses.append(expense)
        return expense

    def settle(self, from_user: str, to_user: str, amount: int) -> None:
        if amount <= 0:
            raise Unbalanced("amount")
        self.settlements.append(Settlement(from_user, to_user, amount))

    def nets(self) -> dict[str, int]:
        """Positive means the group owes this person. Negative means they owe the group."""
        net: dict[str, int] = {}
        for expense in self.expenses:
            total = sum(expense.shares.values())
            net[expense.payer_id] = net.get(expense.payer_id, 0) + total
            for user, amount in expense.shares.items():
                net[user] = net.get(user, 0) - amount
        for settlement in self.settlements:
            net[settlement.from_user] = net.get(settlement.from_user, 0) + settlement.amount
            net[settlement.to_user] = net.get(settlement.to_user, 0) - settlement.amount
        return {user: amount for user, amount in net.items() if amount != 0}

    def simplified(self) -> list[tuple[str, str, int]]:
        """Display only. Same nets, fewer edges. Does not delete history."""
        debtors: list[list] = []
        creditors: list[list] = []
        for user, amount in self.nets().items():
```

```
            if amount < 0:
                debtors.append([user, -amount])
            elif amount > 0:
                creditors.append([user, amount])
        payments: list[tuple[str, str, int]] = []
        i = j = 0
        while i < len(debtors) and j < len(creditors):
            pay = min(debtors[i][1], creditors[j][1])
            payments.append((debtors[i][0], creditors[j][0], pay))
            debtors[i][1] -= pay
            creditors[j][1] -= pay
            if debtors[i][1] == 0:
                i += 1
            if creditors[j][1] == 0:
                j += 1
        return payments
```

test_volume2.py

The checks that were run. The last line of a good run is: all tests passed.

```
"""Run from this directory: python test_volume2.py"""

from __future__ import annotations

import threading

from clocks import FencedStore, LamportClock, LeaseLock, VectorClock, concurrent, happened_before
from codec import decode, encode
from elevator import Elevator
from ledger import GroupLedger, Unbalanced
from logger import ListAppender, Logger
from lru import LruCache
from parking import Full, HourlyFee, ParkingLot, Row, Spot, Vehicle
from raft_model import Cluster, Entry, RaftNode
from rate_limiter import RateLimiter, TokenBucket
from storage import LsmTree, Mvcc, PageStore
from tickets import BadState, Show, Unavailable
from tinylink import (
    ClickQueue,
    Conflict,
    CreateCommand,
    Expired,
    IdSource,
    LinkService,
    MemoryCache,
    MemoryRepo,
    NotFound,
    shard_for,
)


def test_lru() -> None:
    cache = LruCache(2)
    cache.put("a", 1)
    cache.put("b", 2)
    assert cache.get("a") == 1
    cache.put("c", 3)
    assert cache.get("b") is None
    assert cache.get("a") == 1
    cache.put("a", 9)
    cache.put("d", 4)
    assert cache.get("c") is None
    assert cache.get("a") == 9
    assert len(cache) == 2


def test_rate_limiter() -> None:
    bucket = TokenBucket(capacity=2, refill_per_second=1, now=0)
    assert bucket.allow(0)
    assert bucket.allow(0)
    assert not bucket.allow(0)
    assert bucket.allow(1.0)
    limiter = RateLimiter(1, 0)
    assert limiter.allow("priya", 0)
    assert not limiter.allow("priya", 0)
    assert limiter.allow("arun", 0)


def test_codec_and_tinylink() -> None:
    assert decode(encode(0)) == 0
    assert decode(encode(3844)) == 3844
    service = LinkService(MemoryRepo(), MemoryCache(), IdSource(), ClickQueue())
    link = service.create(CreateCommand("https://example.com/a", "tok-1"), now=10)
    again = service.create(CreateCommand("https://example.com/a", "tok-1"), now=11)
```

```
    assert again.code == link.code
    assert service.open(link.code, now=12) == "https://example.com/a"
    try:
        service.create(CreateCommand("https://example.com/b", "tok-2", custom_code=link.code), now=13)
        raise AssertionError("expected conflict")
    except Conflict:
        pass
    try:
        service.create(CreateCommand("notaurl", "tok-3"), now=13)
        raise AssertionError("expected bad url")
    except ValueError:
        pass
    short = service.create(
        CreateCommand("https://example.com/c", "tok-4", expires_at=20),
        now=14,
    )
    service.cache.fail = True
    assert service.open(short.code, now=15) == "https://example.com/c"
    try:
        service.open(short.code, now=20)
        raise AssertionError("expected expired")
    except Expired:
        pass
    try:
        service.open("missing", now=15)
        raise AssertionError("expected missing")
    except NotFound:
        pass
    assert shard_for(link.code, 4) == link.id % 4


def test_parking() -> None:
    spots = [Spot(f"s{i}", "C") for i in range(6)] + [Spot("m0", "M")]
    lot = ParkingLot([Row(spots)], HourlyFee(10))
    bus = lot.park(Vehicle("BUS1", "bus"), now=0)
    assert len(bus.spot_ids) == 5
    bike = lot.park(Vehicle("BIKE", "motorcycle"), now=0)
    assert bike.spot_ids == ["m0"]
    try:
        lot.park(Vehicle("BUS2", "bus"), now=0)
        raise AssertionError("expected full")
    except Full:
        pass
    assert lot.leave(bus.ticket_id, now=3601) == 20
    assert lot.leave(bus.ticket_id, now=99999) == 20
    car = lot.park(Vehicle("CAR", "car"), now=10)
    assert car.spot_ids[0].startswith("s")


def test_tickets() -> None:
    show = Show(["A1", "A2"])
    booking = show.hold(["A1"], "priya", now=0)
    try:
        show.hold(["A1"], "arun", now=1)
        raise AssertionError("expected unavailable")
    except Unavailable:
        pass
    show.confirm(booking.booking_id, now=10)
    assert show.confirm(booking.booking_id, now=11).state == "CONFIRMED"
    held = show.hold(["A2"], "arun", now=0, ttl=5)
    assert show.expire(now=5) == 1
    try:
        show.confirm(held.booking_id, now=6)
        raise AssertionError("expected bad state")
    except BadState:
        pass

    race = Show(["Z"])
    wins: list[str] = []

    def grab(user: str) -> None:
```

```
        try:
            wins.append(race.hold(["Z"], user, now=0).user_id)
        except Unavailable:
            pass

    threads = [threading.Thread(target=grab, args=(name,)) for name in ("priya", "arun")]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
    assert wins == ["priya"] or wins == ["arun"]
    assert len(wins) == 1


def test_elevator() -> None:
    car = Elevator()
    car.call(3, "UP")
    for _ in range(3):
        car.step()
    assert car.floor == 3
    car.step()
    assert car.doors_open
    assert car.direction == "IDLE"

    car = Elevator()
    car.choose(5)
    car.step()
    car.step()
    car.call(1, "DOWN")
    while car.floor != 5:
        car.step()
    car.step()
    assert car.doors_open and car.floor == 5
    while not (car.floor == 1 and car.doors_open):
        car.step()
    assert car.floor == 1


def test_logger() -> None:
    first = ListAppender()
    second = ListAppender()
    second.fail = True
    log = Logger("api", "INFO", [first, second])
    log.log("DEBUG", "hidden")
    log.log("ERROR", "disk", code="aZ")
    assert len(first.events) == 1
    assert first.events[0].context["code"] == "aZ"
    assert log.dropped_errors == 1


def test_ledger() -> None:
    group = GroupLedger()
    group.add_expense("anu", {"anu": 50, "ben": 50})
    assert group.nets() == {"anu": 50, "ben": -50}
    assert group.simplified() == [("ben", "anu", 50)]
    group.settle("ben", "anu", 50)
    assert group.nets() == {}
    try:
        group.add_expense("anu", {"anu": -1})
        raise AssertionError("expected unbalanced")
    except Unbalanced:
        pass


def test_raft() -> None:
    cluster = Cluster(["a", "b", "c"])
    assert cluster.elect("a")
    assert cluster.nodes["a"].role == "leader"
    assert cluster.propose("a", "seat-1") == 0
    for node in cluster.nodes.values():
        assert [entry.command for entry in node.log] == ["seat-1"]
```

```
        assert node.commit_index == 0

    blocked = Cluster(["a", "b", "c"])
    assert not blocked.elect("a", voters=["a"])
    assert blocked.nodes["a"].role != "leader"

    fresh = RaftNode("b")
    fresh.current_term = 1
    fresh.log = [Entry(1, "newer")]
    term, granted = fresh.request_vote(2, "a", last_log_index=-1, last_log_term=0)
    assert term == 2 and not granted


def test_storage_and_clocks() -> None:
    store = PageStore()
    store.put("k", "v1")
    store.crash_after_log("k", "v2")
    assert store.pages["k"] == "v1"
    store.recover()
    assert store.pages["k"] == "v2"

    tree = LsmTree()
    tree.put("k", "old")
    tree.flush()
    tree.put("k", "new")
    assert tree.get("k") == "new"
    tree.delete("k")
    tree.flush()
    assert tree.get("k") is None
    tree.compact()
    assert tree.get("k") is None
    assert all(value is not None for run in tree.runs for value in run.values())

    db = Mvcc()
    t1, snap1 = db.begin()
    db.write(t1, "a", "one")
    t_early, snap_early = db.begin()
    db.commit(t1)
    t2, snap2 = db.begin()
    assert db.read(t_early, snap_early, "a") is None
    assert db.read(t2, snap2, "a") == "one"
    assert db.read(t1, snap1, "a") == "one"

    lamport = LamportClock()
    assert lamport.tick() == 1
    assert lamport.receive(5) == 6

    nodes = ["a", "b"]
    va = VectorClock("a", nodes)
    vb = VectorClock("b", nodes)
    first = va.tick()
    second = vb.receive(first)
    assert happened_before(first, second)
    side = vb.tick()
    other = va.tick()
    assert concurrent(side, other)

    lock = LeaseLock()
    store_fence = FencedStore()
    first_token = lock.acquire("old", now=0, ttl=5)
    assert lock.acquire("new", now=1, ttl=5) is None
    second_token = lock.acquire("new", now=5, ttl=5)
    assert first_token is not None and second_token is not None
    store_fence.write(second_token, "fresh")
    try:
        store_fence.write(first_token, "late")
        raise AssertionError("expected stale token")
    except Exception as exc:
        assert exc.__class__.__name__ == "StaleFencingToken"
    assert store_fence.value == "fresh"
```

```
if __name__ == "__main__":
    test_lru()
    test_rate_limiter()
    test_codec_and_tinylink()
    test_parking()
    test_tickets()
    test_elevator()
    test_logger()
    test_ledger()
    test_raft()
    test_storage_and_clocks()
    print("all tests passed")
```